Lecture Notes on Advanced Algorithms and Complexity

Analysis of Iterative and Recursive Algorithms

Author: ***Abdelkrim Mebarki***
Date : ***October 24, 2026***
Master's Program in Computer Science – All Options
Department of Computer Science – Faculty of Mathematics and Computer Science
University of Science and Technology of Oran – Mohamed Boudiaf (USTO-MB)

1. Lecture Overview

This lecture continues the study of time complexity started in Lectures 1 and 2. Lecture 2 introduced the asymptotic notations O, Ω, and Θ and applied them to short loop fragments. Here, the analysis is extended to complete algorithms: loops whose bounds depend on one another, algorithms whose cost depends on the particular input, and recursive algorithms, whose running time is naturally described by a recurrence relation.

The central goal is to give students a systematic toolbox: (i) rules that turn the structure of an iterative algorithm into a cost function; (ii) the distinction between best-case, worst-case, and average-case analysis; (iii) a method to translate a recursive algorithm into a recurrence relation; and (iv) several techniques — iteration, recursion tree, substitution, and the Master Theorem — to solve that recurrence.

Learning objectives

- Derive the time complexity of iterative algorithms built from simple, nested, dependent, and logarithmic loops.
- Combine the costs of sequences, conditionals, and loops using the sum and product rules.
- Distinguish **best-case**, **worst-case**, and **average-case** running times, and explain how they differ from O, Ω, and Θ.
- Explain the structure of a recursive algorithm (base case, recursive case) and why its cost is described by a recurrence.
- Formulate a recurrence relation from the pseudocode of a recursive algorithm.
- Solve simple recurrences by iteration, recursion tree, and substitution, and apply the **Master Theorem**.
- Analyze classical recursive algorithms: factorial, binary search, merge sort, fast exponentiation, Towers of Hanoi, and Fibonacci.
- Compare iterative and recursive solutions to the same problem in terms of time, memory, and clarity.

Topics covered

- Complexity analysis of iterative algorithms
- Analysis of loops: simple, nested, dependent, and logarithmic loops
- Analysis of recursive functions
- Best-case, worst-case, and average-case analysis
- Introduction to recurrence relations
- Formulating recurrence relations from recursive algorithms
- Solving simple recurrence relations
- Analysis of algorithmic complexity using recurrence relations

Available on zenodo – DOI : 10.5281/zenodo.23617401

- Comparison of iterative and recursive solutions
- Time-complexity analysis of recursive algorithms

Teaching emphasis: *Students should not apply formulas blindly. For every loop or recursive call they should state how many times the repeated work occurs and how large the remaining problem is. The complexity class is a consequence of those two facts.*

2. From Counting Operations to Structural Analysis

Counting every elementary operation, as in Lecture 2, quickly becomes impractical for real algorithms. Fortunately, the cost of an algorithm is built from its control structure, and each structure has a simple composition rule. Once every basic statement is known to cost $\Theta(1)$, an algorithm can be analyzed from the inside out.

Structure	Cost rule	Example
Elementary statement	$\Theta(1)$	$x \leftarrow y + 1$
Sequence S_1 ; S_2	Costs add: $T_1(n) + T_2(n)$; the dominant term wins	$\Theta(n)$ followed by $\Theta(n^2)$ costs $\Theta(n^2)$
Conditional *if C then* S_1 *else* S_2	cost(C) + max(T_1, T_2) for the worst case	A test followed by a $\Theta(1)$ branch or a $\Theta(n)$ branch costs $\Theta(n)$ in the worst case
Loop	Sum, over all iterations, of the cost of the body	n iterations of a $\Theta(1)$ body cost $\Theta(n)$
Nested loops	Product if the inner cost does not depend on the outer counter; otherwise a sum over the outer counter	$n \times n$ iterations cost $\Theta(n^2)$
Function call	Cost of the callee for the actual argument	Calling SUM(A) inside a loop adds $\Theta(n)$ per call

Important:
Sum rule*: $\Theta(f(n)) + \Theta(g(n)) = \Theta(\max(f(n), g(n)))$.*
Product rule*: running a $\Theta(g(n))$ body $\Theta(f(n))$ times costs $\Theta(f(n) \cdot g(n))$, but only if the cost of the body is the same at every iteration.*
Otherwise *the costs must be summed iteration by iteration.*

3. Analysis of Iterative Algorithms

An iterative algorithm repeats work through loops. Its running time is the total cost of all executions of the loop bodies, so the analysis reduces to one question: *how many times is each statement executed, as a function of n?*

3.1 Simple loops

A simple loop is a single loop whose body costs $\Theta(1)$ and whose counter changes by a constant amount at each iteration.

```
for i = 1 to n:
    process(A[i])
```

The body executes n times, so the cost is $\Theta(n)$. Changing the stride or the range by a constant factor does not change the order of growth:

```
for i = 1 to n step 3:     # [n/3] iterations  -> Θ(n)
    process(A[i])
for i = 1 to n/2:          # n/2 iterations     -> Θ(n)
    process(A[i])
for i = 1 to 100:          # 100 iterations     -> Θ(1)
    process(A[i])
```

The last loop shows why one must never assume that every loop is linear: what matters is whether the number of iterations depends on n.

3.2 Consecutive loops and conditionals

Consecutive statements add their costs; the most expensive one determines the order of growth.

```
for i = 1 to n:            # Θ(n)
    process(A[i])
for i = 1 to n:            # Θ(n²)
    for j = 1 to n:
        process(i, j)
```

The total is $\Theta(n) + \Theta(n^2) = \Theta(n^2)$. For a conditional statement, the worst-case analysis keeps the more expensive branch, whereas the best-case analysis keeps the cheaper one (Section 4).

```
if A[0] == x:              # test: Θ(1)
    return True            # branch 1: Θ(1)
else:
    for i = 1 to n:        # branch 2: Θ(n)
        process(A[i])
```

3.3 Nested loops with independent bounds

When the bounds of the inner loop do not depend on the counter of the outer loop, the iteration counts multiply. For two loops running n and m times, the body executes $n \cdot m$ times and the cost is $\Theta(n \cdot m)$. If both bounds equal n, this gives $\Theta(n^2)$.

Example — matrix multiplication of two $n \times n$ matrices:

```
MATRIX-MULTIPLY(A, B, C)
    for i ← 0 to n−1 do
        for j ← 0 to n−1 do
            C[i][j] ← 0
            for k ← 0 to n−1 do
                C[i][j] ← C[i][j] + A[i][k] · B[k][j]
```

The innermost statement executes $n \cdot n \cdot n = n^3$ times, and the initialization executes n^2 times. The total is $\Theta(n^3) + \Theta(n^2) = \Theta(n^3)$.

3.4 Dependent loops

A loop is **dependent** when the bound or the step of an inner loop depends on the counter of an outer loop. The product rule no longer applies, because the cost of the inner loop changes with the outer counter i. The cost must be written as a sum:

$$T(n) = \Sigma_{i=1}^{n} \text{(cost of the inner loop when the outer counter is } i\text{)}$$

Evaluating such sums requires a small set of standard results.

Summation	Closed form	Order of growth
$\Sigma_{i=1}^{n} 1$	n	$\Theta(n)$
$\Sigma_{i=1}^{n} i$	$n(n+1)/2$	$\Theta(n^2)$
$\Sigma_{i=1}^{n} i^2$	$n(n+1)(2n+1)/6$	$\Theta(n^3)$
$\Sigma_{i=0}^{k} 2^i$	$2^{k+1} - 1$	$\Theta(2^k)$
$\Sigma_{i=1}^{n} 1/i$ (harmonic number H_n)	$\ln n + O(1)$	$\Theta(\log n)$
$\Sigma_{i=1}^{n} \log i = \log(n!)$	$\approx n \log n - n \log e$	$\Theta(n \log n)$

Example 1 — triangular loop (selection sort). The inner loop starts after the outer counter, so its length decreases:

```
SELECTION-SORT(A)
    for i ← 0 to n−2 do
        min ← i
        for j ← i+1 to n−1 do
            if A[j] < A[min] then min ← j
        swap A[i] and A[min]
```

For a given i, the comparison $A[j] < A[min]$ executes $n - 1 - i$ times. The total number of comparisons is

$$(n - 1) + (n - 2) + \ldots + 1 = n(n - 1)/2 = \Theta(n^2)$$

This count is the same for every input of size n.

Example 2 — inner bound i^2.

```
for i = 1 to n:
    for j = 1 to i*i:
        process(i, j)
```

The body executes $\mathbf{\Sigma}\, i^2 = n(n + 1)(2n + 1)/6$ times, so the cost is $\mathbf{\Theta}(n^3)$. Although the algorithm contains only two loops, its complexity is cubic.

Example 3 — three dependent loops.

```
for i = 1 to n:
    for j = 1 to i:
        for k = 1 to j:
            process(i, j, k)
```

For fixed i, the two inner loops execute $1 + 2 + \ldots + i = i(i + 1)/2$ times. Summing over i gives $n(n + 1)(n + 2)/6 = \Theta(n^3)$.

Example 4 — the step depends on the outer counter.

```
for i = 1 to n:
    j = 1
    while j <= n:
        process(i, j)
        j = j + i
```

For a given i, the inner loop runs $\lceil n/i \rceil \leq n/i + 1$ times. The total is at most $n \cdot (1 + 1/2 + \ldots + 1/n) + n = n \cdot H_n + n$, which is $\Theta(n \log n)$ because $H_n = \Theta(\log n)$. The cost lies strictly between linear and quadratic.

3.5 Logarithmic loops

When the loop counter is **multiplied** (or **divided**) by a constant $c > 1$ at each iteration, the remaining distance to the bound shrinks geometrically, and the number of iterations is $\Theta(\log n)$.

```
i = 1
while i <= n:
    process(i)
    i = 2 * i
```

After k iterations, $i = 2^k$. The loop stops when $2^k > n$, that is, after $\lfloor \log_2 n \rfloor + 1$ iterations. The same count (up to a constant factor) holds for a counter that starts at n and is halved at each step. The base of the logarithm is irrelevant: $\log_c n = \log_2 n \,/\, \log_2 c$, and a constant factor is absorbed by Θ.

A counter that is **squared** at each iteration is even faster. Starting from $i = 2$, after t iterations $i = 2^{2^t}$, so the loop stops after about $\log_2 \log_2 n$ iterations: $\Theta(\log \log n)$.

A subtle case — a logarithmic loop around a dependent loop.

```
i = 1
while i <= n:
    for j = 1 to i:
        process(j)
    i = 2 * i
```

The outer loop runs $\lfloor \log_2 n \rfloor + 1$ times, and the inner loop is at most n iterations long. This only proves $O(n \log n)$. The exact count is $1 + 2 + 4 + \ldots + 2^k = 2^{k+1} - 1$ with $k = \lfloor \log_2 n \rfloor$, which is smaller than $2n$ and at least n. The tight bound is therefore $\Theta(n)$, not $\Theta(n \log n)$.

> ***Teaching point:*** *"Number of iterations of the outer loop × maximum cost of the inner loop" is only an upper bound. To obtain a tight bound, sum the actual cost of each outer iteration.*

3.6 Summary of loop patterns

Loop pattern	Number of executions of the body	Complexity
i from 1 to n	n	$\Theta(n)$
$i \leftarrow i + c$	$\lceil n/c \rceil$	$\Theta(n)$
$i \leftarrow c \cdot i$ or $i \leftarrow i/c$	$\lfloor \log_c n \rfloor + 1$	$\Theta(\log n)$
$i \leftarrow i^2$	$\approx \log_2 \log_2 n$	$\Theta(\log \log n)$
Two nested loops, independent bounds n and m	$n \cdot m$	$\Theta(n \cdot m)$
Inner loop from 1 to i	$n(n+1)/2$	$\Theta(n^2)$
Inner loop with step i	$\Sigma \lceil n/i \rceil$	$\Theta(n \log n)$
Outer counter doubles, inner loop from 1 to i	$2^{k+1} - 1$, $k = \lfloor \log_2 n \rfloor$	$\Theta(n)$

4. Best-Case, Worst-Case, and Average-Case Analysis

4.1 Why the input instance matters

Until now, the cost of an algorithm has been written as a function of n alone. For many algorithms, however, two inputs of the same size can lead to very different amounts of work. Let I_n denote the set of valid inputs of size n, and let **cost(x)** be the number of elementary operations performed on input x.

Case	Definition	Meaning
Best case	$T_{best}(n) = \min \{ \text{cost}(x) : x \text{ is an input of size } n \}$	Cost on the most favorable input
Worst case	$T_{worst}(n) = \max \{ \text{cost}(x) : x \text{ is an input of size } n \}$	Cost on the most unfavorable input; a guarantee for all inputs
Average case	$T_{avg}(n) = \Sigma \Pr(x) \cdot \text{cost}(x)$, summed over all inputs of size n	Expected cost under an explicitly stated probability distribution

The average case is meaningful only when the distribution of the inputs is stated. A common and convenient assumption is that all inputs of size n are equally likely, or that all orderings of the data are equally likely.

4.2 Example: linear search

```
LINEAR-SEARCH(A, x)
    for i ← 0 to n−1 do
        if A[i] = x then return i
    return −1
```

- **Best case:** x is at position **0**; one comparison; **$\Theta(1)$**.
- **Worst case:** x is at the last position or absent; n comparisons; **$\Theta(n)$**.
- **Average case:** assume x is present and equally likely to be at each of the n positions. A search for position i costs i comparisons, so the expected number is $(1 + 2 + \ldots + n)/n = (n+1)/2$. If x is absent with probability q, the expected cost is $q \cdot n + (1 - q) \cdot (n+1)/2$. This is $\Theta(n)$ for every fixed q.

4.3 Example: insertion sort

```
INSERTION-SORT(A)
    for i ← 1 to n−1 do
        key ← A[i]
        j ← i − 1
        while j ≥ 0 and A[j] > key do
            A[j+1] ← A[j]
            j ← j − 1
        A[j+1] ← key
```

The outer loop always runs $n - 1$ times. The cost depends on how far each element must move to the left, that is, on how many earlier elements are larger than it. A pair of positions (p, q) with $p < q$ and $A[p] > A[q]$ is called an **inversion**; the total number of shifts equals the number of inversions.

Case	Input	Comparisons	Complexity
Best	Already sorted: no inversions	$n - 1$ (one per iteration)	$\Theta(n)$
Worst	Strictly decreasing order: every pair is an inversion	$1 + 2 + \ldots + (n - 1) = n(n - 1)/2$	$\Theta(n^2)$
Average	All $n!$ orderings equally likely	About $n(n - 1)/4$ shifts plus at most $n - 1$ extra comparisons	$\Theta(n^2)$

The average number of inversions is $n(n - 1)/4$ because each of the $n(n - 1)/2$ pairs of positions is inverted with probability $1/2$ in a random ordering. Insertion sort also adapts to its input: it is fast on nearly sorted data.

4.4 Cases are not notations

Important: *A frequent confusion is* ***"O describes the worst case and Ω the best case"****. This is false. The* ***case*** *says which inputs are considered; the* ***notation*** *says how a function is bounded.*
Each case has its own cost function $(T_{best}, T_{worst}, T_{avg})$*, each of these functions can be described with* O, Ω, *or* Θ.

Statement about insertion sort	True	Explanation
worst-case time is $\Theta(n^2)$	Yes	The decreasing input costs $n(n - 1)/2$ comparisons, and no input costs more.
Best-case time is $\Theta(n)$	Yes	The sorted input costs $n - 1$ comparisons, and no input costs less.
Running time is $O(n^2)$ on every input	Yes	No input exceeds $n(n - 1)/2$ comparisons.
Running time is $\Omega(n)$ on every input	Yes	Every input needs at least $n - 1$ comparisons.
Running time is $\Omega(n^2)$ on every input	No	The sorted input needs only $n - 1$ comparisons.
Running time is $\Theta(n^2)$ on every input	No	The bound is tight for the worst case only.

4.5 Which case should be used?

- **Worst case:** the default choice. It gives a guarantee that holds for every input and does not require assumptions about the data.
- **Average case:** closer to typical behavior, but only as reliable as the assumed input distribution, and often harder to compute.
- **Best case:** rarely sufficient on its own, but useful to identify which input structure an algorithm exploits (for example, insertion sort on sorted data).

Teaching point: *The average case is* ***not*** *the average of the best and worst cases. It is an expected value over a probability distribution of inputs.*

5. Analysis of Recursive Algorithms

5.1 Structure of a recursive algorithm

A **recursive algorithm** solves a problem by reducing it to one or more smaller instances of the *same* problem. Every correct recursive algorithm has two ingredients:

- **Base case(s):** instances small enough to be solved directly, without further recursion.
- **Recursive case:** a rule that reduces the current instance to smaller instances and combines their results. Each call must make progress toward a base case; otherwise the recursion never terminates.

Example — factorial. $n! = n \cdot (n - 1)!$ for $n > 1$, and $1! = 1$.

```
FACT(n)
    if n ≤ 1 then return 1
    else return n · FACT(n − 1)
```

The calls unfold as a chain, and the results are combined when the calls return:

```
FACT(4) = 4 · FACT(3)
          FACT(3) = 3 · FACT(2)
                    FACT(2) = 2 · FACT(1)
                              FACT(1) = 1          (base case)
                    FACT(2) = 2 · 1 = 2
          FACT(3) = 3 · 2 = 6
FACT(4) = 4 · 6 = 24
```

5.2 Why counting loops is not enough

In an iterative algorithm, repeated work is visible as loops. In a recursive algorithm, the repetition is hidden inside the calls, and the cost of a call on an instance of size n **includes the cost of the calls on smaller instances**. The running time $\boldsymbol{T(n)}$ therefore appears on both sides of an equation. Such an equation is called a **recurrence relation**.

For **FACT**, one call performs a comparison and a multiplication (constant work) and makes one call on size $\boldsymbol{n-1}$. Hence $\boldsymbol{T(n) = T(n-1) + c}$. The remainder of this lecture explains how to write such equations systematically and how to solve them.

5.3 A four-step method

1. **Define the size** n of the instance (the parameter that decreases in the calls).
2. **Identify the base case** and its cost, usually a constant.
3. **Describe the recursive case:** how many recursive calls are *actually executed* on a given instance, what is the size of each, and how much non-recursive work $f(n)$ (splitting, combining, testing) is done.
4. **Write and solve** the recurrence $T(n)$ = (sum of the costs of the recursive calls) + $f(n)$.

6. Recurrence Relations: Definition and Formulation

6.1 Core vocabulary

Term	Definition
Recurrence relation	An equation that expresses $T(n)$ in terms of the values of T at smaller arguments.
Initial condition/Base case	The value of T for the smallest inputs; it stops the recursion. Usually in algorithm analysis: $\Theta(1)$.
Solving a recurrence	Finding a closed form for $T(n)$, or at least its asymptotic order (O, Ω, Θ).
Iteration (unfolding, expansion)	Repeatedly replacing T on the right-hand side by its own recurrence until the base case appears, then summing the resulting pattern.
Recursion tree	A tree whose nodes are the subproblems; each node is labeled with its non-recursive cost, and the total cost is the sum of all labels.
Substitution method	Guess a bound for $T(n)$, then prove it by mathematical induction.
Master Theorem	A ready-made solution for recurrences of the form $T(n) = a \cdot T(n/b) + f(n)$.

6.2 Two common families

- **Subtractive recurrences:** $T(n) = a \cdot T(n-b) + f(n)$. The instance shrinks by a constant amount (factorial, Towers of Hanoi, naive Fibonacci).
- **Divide-and-conquer recurrences:** $T(n) = a \cdot T(n/b) + f(n)$. The instance is split into a subproblems of size about n/b, and $f(n)$ is the cost of dividing the problem and combining the results (binary search, merge sort, fast exponentiation).

6.3 Formulating a recurrence from an algorithm

Example 1 — factorial. From the pseudocode in Section 5.1: one recursive call on $n-1$ and a constant amount of other work.

```
T(n) = T(n-1) + c      for n > 1
T(1) = c₀
```

Example 2 — recursive binary search.

```
BINARY-SEARCH-REC(A, x, low, high)
    if low > high then return −1
    mid ← low + ⌊(high − low) / 2⌋
    if A[mid] = x then return mid
    else if A[mid] < x then return BINARY-SEARCH-REC(A, x, mid+1, high)
    else return BINARY-SEARCH-REC(A, x, low, mid−1)
```

The code contains two recursive calls, but **only one of them is executed** in any invocation, and it works on at most half of the current interval (n = high − low + 1). The recurrence is:

$$T(n) = T(n/2) + c \quad \text{for } n > 1$$
$$T(1) = c_0$$

Example 3 — merge sort.

```
MERGE-SORT(A, lo, hi)
    if lo < hi then
        mid ← lo + ⌊(hi − lo) / 2⌋
        MERGE-SORT(A, lo, mid)
        MERGE-SORT(A, mid+1, hi)
        MERGE(A, lo, mid, hi)          // merges two sorted halves: Θ(n)
```

Here **both** recursive calls are executed, each on about half of the elements, and the merge step costs $\Theta(n)$ (it examines every element once). The recurrence is:

$$T(n) = 2T(n/2) + cn \quad \text{for } n > 1$$
$$T(1) = c_0$$

6.4 Recurrences of classical recursive algorithms

Algorithm	Recursive calls executed	Size of each call	Other work	Recurrence
Factorial	1	$n - 1$	$\Theta(1)$	$T(n) = T(n-1) + \Theta(1)$
Binary search	1 (out of 2 branches)	$\leq n/2$	$\Theta(1)$	$T(n) = T(n/2) + \Theta(1)$
Merge sort	2	$n/2$	$\Theta(n)$ (merge)	$T(n) = 2T(n/2) + \Theta(n)$
Fast exponentiation	1	$\lfloor n/2 \rfloor$	$\Theta(1)$	$T(n) = T(n/2) + \Theta(1)$
Towers of Hanoi	2	$n - 1$	$\Theta(1)$	$T(n) = 2T(n-1) + \Theta(1)$
Naive Fibonacci	2	$n - 1$ and $n - 2$	$\Theta(1)$	$T(n) = T(n-1) + T(n-2) + \Theta(1)$

6.5 A remark on floors and ceilings

Real recurrences often contain floors and ceilings, such as $T(\lfloor n/2 \rfloor)$. For the recurrences studied in this lecture, ignoring them, or assuming that n is a power of b, does not change the asymptotic order of the solution. We therefore write $n/2$ for simplicity. A rigorous justification belongs to a more advanced treatment of recurrences.

7. Solving Recurrence Relations

Four techniques are used in this course. The first two build intuition; the third proves a bound that was guessed; the fourth gives the answer directly for a large family of divide-and-conquer recurrences. In all examples, c and c_0 denote positive constants.

7.1 The iteration method

Replace T on the right-hand side by its definition, again and again, until the base case is reached; then identify the pattern and sum it.

(a) $T(n) = T(n-1) + c$, $T(1) = c_0$

```
T(n) = T(n-1) + c
     = T(n-2) + 2c
     = T(n-3) + 3c
     ...
     = T(n-k) + k·c            take k = n-1
     = T(1) + (n-1)·c = c₀ + (n-1)·c = Θ(n)
```

(b) $T(n) = T(n-1) + n$, $T(1) = 1$

```
T(n) = T(n-1) + n
     = T(n-2) + (n-1) + n
     = T(n-3) + (n-2) + (n-1) + n
     ...
     = T(1) + 2 + 3 + ... + n = n(n+1)/2 = Θ(n²)
```

(c) $T(n) = T(n/2) + c$, $T(1) = c_0$, with $n = 2^k$

```
T(n) = T(n/2) + c
     = T(n/4) + 2c
     = T(n/8) + 3c
     ...
     = T(n/2^k) + k·c          with k = log₂ n
     = T(1) + c·log₂ n = Θ(log n)
```

(d) $T(n) = 2T(n/2) + n$, $T(1) = 1$, with $n = 2^k$

```
T(n) = 2T(n/2) + n
     = 2[2T(n/4) + n/2] + n     = 4T(n/4) + 2n
     = 4[2T(n/8) + n/4] + 2n    = 8T(n/8) + 3n
     ...
     = 2^k · T(n/2^k) + k·n      with k = log₂ n
     = n·T(1) + n·log₂ n = n + n·log₂ n = Θ(n log n)
```

(e) $T(n) = 2T(n-1) + 1$, $T(0) = 0$

```
T(n) = 2T(n-1) + 1
     = 2[2T(n-2) + 1] + 1      = 4T(n-2) + 2 + 1
     = 4[2T(n-3) + 1] + 2 + 1  = 8T(n-3) + 4 + 2 + 1
     ...
     = 2^n · T(0) + (2^(n-1) + ... + 2 + 1) = 2^n – 1 = Θ(2^n)
```

7.2 The recursion-tree method

The recursion tree displays the subproblems level by level. The cost of the algorithm is the sum, over all levels, of the cost of the nodes of that level. For $T(n) = 2T(n/2) + n$, the root costs n, it has two children of size $n/2$, each with cost $n/2$, and so on:

Level i	Number of subproblems	Size of each	Cost of the level
0	1	n	n
1	2	$n/2$	$2 \cdot n/2 = n$
2	4	$n/4$	$4 \cdot n/4 = n$
i	2^i	$n/2^i$	n
$\log_2 n$ (leaves)	n	1	n

There are $\log_2 n + 1$ levels, each costing n, so $T(n) = n(\log_2 n + 1) = \Theta(n \log n)$.

The shape of the cost per level reveals three typical regimes, which explain the Master Theorem of Section 7.4:

Recurrence	Cost of level i	Behavior	Total
$T(n) = 4T(n/2) + n$	$4^i \cdot (n/2^i) = 2^i \cdot n$	Grows geometrically: the leaves dominate ($4^{\log_2 n} = n^2$ leaves)	$\Theta(n^2)$

Recurrence	Cost of level *i*	Behavior	Total
$T(n) = 2T(n/2) + n$	n	Constant per level: all levels contribute equally	$\Theta(n \log n)$
$T(n) = 2T(n/2) + n^2$	$2^i \cdot (n/2^i)^2 = n^2/2^i$	Shrinks geometrically: the root dominates	$\Theta(n^2)$

7.3 The substitution method

The substitution method proves a bound that has been *guessed*, typically from the iteration or the recursion tree. It has three steps: (1) state the guess, (2) assume it holds for all smaller arguments (induction hypothesis), and (3) prove that it holds for n, including the base case.

Claim. If $T(n) = 2T(n/2) + n$ with $T(1) = 1$ and n a power of 2, then $T(n) \le 2n \log_2 n$ for every $n \ge 2$.

```
Base case (n = 2):  T(2) = 2T(1) + 2 = 4  ≤  2·2·log₂ 2 = 4.   OK

Inductive step (n ≥ 4): assume T(n/2) ≤ 2·(n/2)·log₂(n/2). Then
    T(n) = 2T(n/2) + n
         ≤ 2·2·(n/2)·log₂(n/2) + n
         = 2n·(log₂ n − 1) + n
         = 2n·log₂ n − n
         ≤ 2n·log₂ n.      OK
```

The same argument with the hypothesis $T(n) \ge n \log_2 n$ proves the lower bound, so $T(n) = \Theta(n \log n)$.

- **Why the base case is $n = 2$ and not $n = 1$:** for $n = 1$ the guess gives $2 \cdot 1 \cdot \log_2 1 = 0$, whereas $T(1) = 1$. The bound is required only for $n \ge n_0$, so the induction starts at $n_0 = 2$.
- **A wrong guess fails visibly:** assuming $T(n) \le cn$ leads to $T(n) \le 2c(n/2) + n = cn + n$, which is not $\le cn$. The induction breaks, and indeed $T(n)$ is not $O(n)$.
- **The conclusion must have exactly the form of the hypothesis.** Ending with "$\le cn + n$" and declaring it "$= O(n)$" is invalid, because the constant c would change at each level of the induction.
- **Sometimes the hypothesis must be strengthened.** For $T(n) = 2T(n/2) + 1$, the guess $T(n) \le cn$ fails, but the stronger guess $T(n) \le cn - 1$ works: $2(c \cdot n/2 - 1) + 1 = cn - 1$.

7.4 The Master Theorem

The Master Theorem gives the solution of divide-and-conquer recurrences directly. Define the **critical exponent** $E = \log_b a$. The quantity n^E is the number of leaves of the recursion tree, and it is the benchmark against which the non-recursive cost $f(n)$ is compared.

Master Theorem

Let $a \ge 1$ and $b > 1$ be constants, let $f(n)$ be an asymptotically positive function, and let $T(n) = a \cdot T(n/b) + f(n)$. Then $T(n)$ has the following asymptotic bounds.

Case	Condition on $f(n)$	Solution	What dominates
1	$f(n) = O(n^{E-\varepsilon})$ for some constant $\varepsilon > 0$ (f is polynomially smaller than n^E)	$T(n) = \Theta(n^E)$	The leaves
2	$f(n) = \Theta(n^E \log^k n)$ for some constant $k \ge 0$	$T(n) = \Theta(n^E \log^{k+1} n)$	All levels contribute comparably
3	$f(n) = \Omega(n^{E+\varepsilon})$ for some constant $\varepsilon > 0$ (f is polynomially larger), **and** the regularity condition $a \cdot f(n/b) \le c \cdot f(n)$ holds for some constant $c < 1$ and all large n	$T(n) = \Theta(f(n))$	The root

How to apply it. (1) Identify a, b, and $f(n)$. (2) Compute $E = \log_b a$ and the benchmark n^E. (3) Compare $f(n)$ with n^E. (4) In case 3, check the regularity condition; it holds automatically for polynomial functions such as $f(n) = n^k$.

Recurrence	a, b	$E = \log_b a$	Comparison of $f(n)$ with n^E	Case	Solution
$T(n) = 2T(n/2) + n$	2, 2	1	$f = \Theta(n^1)$: same order	2 ($k = 0$)	$\Theta(n \log n)$
$T(n) = T(n/2) + 1$	1, 2	0	$f = \Theta(n^0)$: same order	2 ($k = 0$)	$\Theta(\log n)$
$T(n) = 2T(n/2) + 1$	2, 2	1	$f = 1 = O(n^{1-\varepsilon})$: polynomially smaller	1	$\Theta(n)$
$T(n) = 4T(n/2) + n$	4, 2	2	$f = n = O(n^{2-\varepsilon})$: polynomially smaller	1	$\Theta(n^2)$
$T(n) = 2T(n/2) + n^2$	2, 2	1	$f = n^2 = \Omega(n^{1+\varepsilon})$; $2(n/2)^2 = n^2/2 \le \frac{1}{2}n^2$	3	$\Theta(n^2)$
$T(n) = 3T(n/2) + n^2$	3, 2	≈ 1.585	$f = n^2 = \Omega(n^{1.585+\varepsilon})$; $3(n/2)^2 = \frac{3}{4}n^2$	3	$\Theta(n^2)$
$T(n) = T(n/2) + n$	1, 2	0	$f = n = \Omega(n^{0+\varepsilon})$; $n/2 = \frac{1}{2}n$	3	$\Theta(n)$
$T(n) = 2T(n/2) + n \log n$	2, 2	1	$f = \Theta(n \log^1 n)$	2 ($k = 1$)	$\Theta(n \log^2 n)$

Intuition. The recursion tree has depth $\log_b n$; level i contains a^i subproblems of size n/b^i, and the last level, at depth $k = \log_b n$, contains $a^k = n^E$ leaves. If the cost per level decreases geometrically from the root (case 3), the root dominates; if it increases geometrically (case 1), the leaves dominate; if it stays about the same (case 2), the cost of one level is multiplied by the number of levels, $\log n$.

When the Master Theorem does not apply

- **Subtractive recurrences** such as $T(n) = T(n - 1) + n$: the instance does not shrink by a constant factor b. Use iteration (Section 7.1).
- **The number of subproblems is not a constant** ≥ 1, for example $T(n) = 2^n \cdot T(n/2) + n^n$.
- **Gap between the cases:** for $T(n) = 2T(n/2) + n/\log n$, the function f is smaller than n^E but not *polynomially* smaller, and it is not of the form $n^E \log^k n$ with $k \ge 0$. Use the recursion tree or substitution instead.
- **Case 3 without regularity:** the condition $a \cdot f(n/b) \le c \cdot f(n)$ must be verified when f is not a simple polynomial.

Teaching point →
*Compare f(n) with n^E by their polynomial growth, not by a quick glance at the logarithmic factors. The function n log n is **not** polynomially larger than n; that is why 2T(n/2) + n log n falls under case 2 and not case 3.*

7.5 Summary of standard recurrences

Recurrence	Solution	Typical algorithm
$T(n) = T(n - 1) + \Theta(1)$	$\Theta(n)$	Factorial, recursive scan of an array
$T(n) = T(n - 1) + \Theta(n)$	$\Theta(n^2)$	Recursive selection sort
$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$	Binary search, fast exponentiation
$T(n) = T(n/2) + \Theta(n)$	$\Theta(n)$	Halving with linear work per step
$T(n) = 2T(n/2) + \Theta(1)$	$\Theta(n)$	Recursive maximum by halving, tree traversal
$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$	Merge sort
$T(n) = 2T(n - 1) + \Theta(1)$	$\Theta(2^n)$	Towers of Hanoi
$T(n) = T(n - 1) + T(n - 2) + \Theta(1)$	$\Theta(\phi^n)$, $\phi \approx 1.618$	Naive Fibonacci

8. Time-Complexity Analysis of Recursive Algorithms: Worked Examples

Each example follows the four-step method of Section 5.3: define the size, identify the base case, describe the recursive case, then write and solve the recurrence.

8.1 Factorial

From Section 6.3, $T(n) = T(n-1) + c$ with $T(1) = c_0$.

The iteration of Section 7.1(a) gives $T(n) = \Theta(n)$: the function makes n calls, each performing constant work.

The calls are nested, so up to n of them are waiting at the same time; this stack depth matters when recursion is compared with iteration (Section 9).

8.2 Recursive binary search

From Section 6.3, $T(n) = T(n/2) + c$.

By the Master Theorem, $a = 1$, $b = 2$, $E = \log_2 1 = 0$, and $f(n) = c = \Theta(n^0)$.

This is case **2** with $k = 0$, hence $T(n) = \Theta(\log n)$. The iteration of Section 7.1(c) gives the same result.

- **Best case:** the target is at the first middle position; one comparison; $\Theta(1)$.
- **Worst case:** the target is absent or found at the last possible step; the interval is halved about $\log_2 n$ times; $\Theta(\log n)$.

8.3 Merge sort

The merge step combines two sorted halves containing n elements in total. It repeatedly compares the front elements of both halves and copies the smaller one; it performs at most $n - 1$ comparisons and moves every element once, hence $\Theta(n)$. With the recurrence $T(n) = 2T(n/2) + \Theta(n)$, we have $a = 2$, $b = 2$, $E = 1$, and $f(n) = \Theta(n)$: case **2**, so $T(n) = \Theta(n \log n)$.

The recursion tree of Section 7.2 gives the intuition: $\log_2 n + 1$ levels, each of which merges a total of n elements. The running time does not depend on the input order, so the best, worst, and average cases are all $\Theta(n \log n)$.

n	n^2 operations	$n \log_2 n$ operations	Ratio
1 000	10^6	$\approx 10^4$	≈ 100
1 000 000	10^{12}	$\approx 2 \times 10^7$	$\approx 50\,000$
1 000 000 000	10^{18}	$\approx 3 \times 10^{10}$	$\approx 33\,000\,000$

With the model of Lecture 1 (10^9 elementary operations per second), 10^{12} operations take about 17 minutes, whereas 2×10^7 operations take about 20 milliseconds.

8.4 Fast exponentiation

Computing x^n with the definition requires $n - 1$ multiplications, that is, $\Theta(n)$. Squaring reduces the exponent by half at each call:

```
POWER(x, n)
    if n = 0 then return 1
    y ← POWER(x, ⌊n/2⌋)
    if n is even then return y · y
    else return y · y · x
```

The function performs one recursive call on $\lfloor n/2 \rfloor$ and at most two multiplications: $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$. The recurrence is the same as for binary search, hence the same solution. For $n = 10^9$, the naive method needs about 10^9 multiplications; fast exponentiation needs at most **60**.

Here the cost is measured in multiplications. For very large numbers, the cost of one multiplication grows with the number of digits; this is why the choice of the cost model, discussed in Lecture 1, always has to be stated.

8.5 Towers of Hanoi

The problem consists in moving n disks from a source peg to a target peg, one disk at a time, never placing a larger disk on a smaller one, using a third peg.

```
HANOI(n, from, to, via)
    if n = 0 then return
    HANOI(n − 1, from, via, to)
    move disk n from “from” to “to”
    HANOI(n − 1, via, to, from)
```

Let $T(n)$ be the number of moves. Two recursive calls on $n-1$ disks and one move give $T(n) = 2T(n-1) + 1$ with $T(0) = 0$. The iteration of Section 7.1(e) yields $T(n) = 2^n - 1 = \Theta(2^n)$.

This exponential growth is not a weakness of the algorithm: the solution itself consists of $2^n - 1$ moves, so no algorithm that writes the sequence of moves can be faster. With 64 disks, even 10^9 moves per second would require about 585 years.

8.6 Naive Fibonacci

```
FIB(n)
    if n ≤ 1 then return n
    return FIB(n − 1) + FIB(n − 2)
```

Let $T(n)$ be the number of calls. Then $T(0) = T(1) = 1$ and $T(n) = T(n-1) + T(n-2) + 1$. Since T is non-decreasing, two simple bounds follow.

- **Upper bound:** $T(n-2) \le T(n-1)$, hence $T(n) \le 2T(n-1) + 1$, and as in Section 7.1(e), $T(n) = O(2^n)$.
- **Lower bound:** $T(n-1) \ge T(n-2)$, hence $T(n) \ge 2T(n-2)$, and unfolding $n/2$ times gives:

$$T(n) = \Omega(2^{n/2}) = \Omega((\sqrt{2})^n).$$

The exact solution is $T(n) = 2F_{n+1} - 1$, where F_n is the n-th Fibonacci number. Since $F_n = \Theta(\phi^n)$ with $\phi = (1 + \sqrt{5})/2 \approx 1.618$, we obtain $T(n) = \Theta(\phi^n)$. This lies between the two bounds above, since $\sqrt{2} \approx 1.414 < 1.618 < 2$.

n	Number of calls $2F_{n+1} - 1$	Time at 10^9 calls per second
10	177	under 1 µs
20	21 891	≈ 22 µs
30	2 692 537	≈ 2.7 ms
40	331 160 281	≈ 0.33 s
50	40 730 022 147	≈ 41 s

The explosion has a precise cause: the same subproblems are solved again and again. **FIB($n-2$)** is computed once inside **FIB(n)** and once more inside **FIB($n-1$)**, and the duplication repeats at every level of the tree. This observation is the starting point of the comparison below.

9. Comparison of Iterative and Recursive Solutions

9.1 The same problem, two styles

Fibonacci numbers can also be computed with a loop that keeps only the last two values:

```
FIB-ITER(n)
    if n ≤ 1 then return n
    a ← 0
    b ← 1
    for i ← 2 to n do
        c ← a + b
        a ← b
        b ← c
    return b
```

The loop executes $n-1$ times with constant work per iteration: $\Theta(n)$ additions, using a constant number of variables. Compared with $\Theta(\phi^n)$ for the naive recursion, the improvement is dramatic: for $n = 50$, about **49** iterations instead of **40** billion calls. (Additions are counted as elementary operations here; the numbers F_n themselves grow with n, which is again a question of the cost model.)

Conversely, factorial and binary search have equally efficient recursive and iterative versions. The iterative factorial is a loop of $n - 1$ multiplications, and the iterative binary search was presented in Lecture 1.

9.2 Criteria of comparison

Criterion	Iterative solution	Recursive solution
Repetition mechanism	Loop with a counter or a condition	Function calls on smaller instances
Natural use	Scanning, accumulating, counting	Self-similar structures: trees, divide and conquer, backtracking
Analysis technique	Summation over the iterations	Recurrence relation
Extra memory	Usually a constant number of variables	One stack frame per active call: $\Theta(n)$ for factorial, $\Theta(\log n)$ for binary search
Overhead	Low	Each call costs time (parameter passing, stack frame); a constant factor, not a change of order
Risk	—	Stack overflow if the recursion is very deep
Readability	Can become long for self-similar problems	Often shorter and close to the mathematical definition

Problem	Iterative time	Recursive time	Comment
Factorial	$\Theta(n)$	$\Theta(n)$	Same time; stack of depth n for the recursive version
Binary search	$\Theta(\log n)$	$\Theta(\log n)$	Same time; recursion adds a stack of depth $\Theta(\log n)$
Fibonacci	$\Theta(n)$	$\Theta(\phi^n)$ (naive)	Large gap, caused by repeated subproblems
Power x^n	$\Theta(n)$ (naive loop)	$\Theta(\log n)$ (squaring)	The algorithm matters, not the style

9.3 Converting between the two styles

Every recursive algorithm can be rewritten iteratively by simulating the call stack with an explicit stack. In the special case of **tail recursion**, where the recursive call is the last action of the function, the conversion is immediate: the parameters are simply updated and the function loops. Factorial with an accumulator illustrates this:

```
FACT-TAIL(n, acc)                    FACT-LOOP(n)
    if n ≤ 1 then return acc            acc ← 1
    return FACT-TAIL(n − 1, n·acc)       while n > 1 do
                                           acc ← n · acc
                                           n ← n − 1
                                     return acc
```

A formal treatment of memory usage, including the stack, is the subject of Lecture 8 on space complexity.

Teaching point: *The question is never "is recursion slow?" but "how much work does the algorithm perform?" Naive Fibonacci is slow not because it is recursive, but because it solves the same subproblems again and again. Recursive binary search and recursive merge sort are as efficient as any iterative alternative in running time.*

10. Common Student Mistakes

- Assuming that two nested loops always give $\Theta(n^2)$, even when the inner bound depends on the outer counter or the inner loop is logarithmic.
- Using "number of outer iterations × maximum inner cost" as if it were a tight bound (the doubling loop of Section 3.5 is $\Theta(n)$, not $\Theta(n \log n)$).
- Forgetting that a function called inside a loop contributes its own cost at every iteration.
- Assuming that every loop is linear, including loops with a fixed number of iterations.
- Believing that O means worst case and Ω means best case.
- Computing the average case as the mean of the best and worst cases.
- Writing a recurrence with the wrong number of recursive calls, for example counting both branches of binary search although only one is executed.
- Forgetting the base case, or forgetting the non-recursive work $f(n)$ (the merge step of merge sort, for example).
- Applying the Master Theorem to a recurrence that is not of the form $a \cdot T(n/b) + f(n)$, such as $T(n) = T(n-1) + n$.
- Applying case 3 of the Master Theorem without checking the regularity condition, or comparing $f(n)$ with n^E while ignoring polynomial growth.
- Ending a substitution proof with a different form than the induction hypothesis.
- Concluding that recursive algorithms are slower than iterative ones.

11. A Practical Procedure for Analyzing an Algorithm

Iterative algorithms

1. Define the input-size parameter n and identify the dominant basic operation.
2. For each loop, determine its number of iterations as a function of n and of the counters of the enclosing loops.
3. Write the total cost as a product (independent bounds) or as a sum over the outer counter (dependent bounds).
4. Evaluate the sum with the standard formulas, and add the costs of consecutive blocks.
5. Keep the dominant term and state the bound, specifying the case studied (best, worst, or average).

Recursive algorithms

1. Define the size n and identify the base case and its cost.
2. Count the recursive calls that are actually executed, the size of each, and the non-recursive work $f(n)$.
3. Write the recurrence $T(n)$ = (cost of the recursive calls) + $f(n)$.
4. Choose a method: the Master Theorem for $a \cdot T(n/b) + f(n)$; iteration or recursion tree for subtractive recurrences.
5. If a bound was guessed, verify it by substitution, then state the result.

Recommended habit →
Before computing anything, answer two questions:
1. *how many times is the repeated work performed?*
2. *and how large is the problem that remains?*

The formulas only turn those two answers into a bound.

12. Lecture Summary

The cost of an algorithm follows from its structure. For iterative algorithms, one sums the cost of the loop bodies; for recursive algorithms, one writes and solves a recurrence relation.

- Sequences add costs, conditionals keep the most expensive branch in the worst case, and loops sum the cost of their bodies.
- Independent nested loops multiply; dependent loops require a sum over the outer counter.
- A loop whose counter is multiplied or divided by a constant runs $\Theta(\log n)$ times.
- Best, worst, and average cases describe different sets of inputs; they are independent of the notations O, Ω, and Θ.
- A recursive algorithm has a base case and a recursive case; its running time satisfies a recurrence relation.
- The recurrence is read from the code: number of calls executed, size of each call, and non-recursive work.
- Recurrences are solved by iteration, recursion tree, substitution, or the Master Theorem.
- Merge sort is $\Theta(n \log n)$, binary search is $\Theta(\log n)$, Towers of Hanoi is $\Theta(2^n)$, and naive Fibonacci is $\Theta(\phi^n)$.
- Recursive and iterative solutions can have the same complexity; what matters is the amount of work, not the style.

Take-home message: *To analyze an algorithm, translate its structure into a cost: a sum for loops, a recurrence for recursion. Then explain why the cost grows the way it does.*

13. Transition to the Next Lecture

With the tools of this lecture, the cost of any algorithm built from loops and recursion can be derived. The next lecture applies them to data structures: how the choice of a structure determines the cost of the operations that an algorithm performs on its data, and therefore the complexity of the algorithm as a whole.

14. Further Reading

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: The MIT Press, 2022. ISBN: 978-0-262-04630-5. (Chapters on growth of functions and on solving recurrences, including the Master Theorem.)

[2] J. Kleinberg and É. Tardos, *Algorithm Design*, 1st ed. Boston, MA, USA: Addison-Wesley, 2005. ISBN: 978-0-321-29535-4.

[3] S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani, *Algorithms*, 1st ed. New York, NY, USA: McGraw-Hill, 2006. ISBN: 978-0-07-352340-8.

[4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011. ISBN: 978-0-321-57351-3.

[5] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1994. ISBN: 978-0-201-55802-9. (Summations and recurrences.)

[6] R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, 2nd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013. ISBN: 978-0-321-90575-8. (Average-case analysis.)

[7] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed. Boston, MA, USA: Pearson, 2012. ISBN: 978-0-13-231681-1.